

model-access

User Guide — Model Access (Making Provider Models Available)

Field	Value
Revision	1
Last modified	2026-06-10
Status	TARGET-STATE GUIDE — mixes IMPLEMENTED and PLANNED behaviour; every clause is labelled
Scope	helix_code/ (inner Go app dev.helix.code)+ submodules/ helix_agent(dev.helix.agent)
Source plans	docs/superpowers/specs/plans/2026-06-10-SP1-model-access-plan.md, ...-SP2-helixagent-exposure-plan.md
Authority	Cascades from constitution/ + root CLAUDE.md/CONSTITUTION.md. Anti-bluff §11.4 governs: nothing here is claimed “working” without it being verifiable in the current tree.

Anti-bluff note (§11.4 / §11.4.123). This is a guide to how model access *is meant to work*. Where a behaviour is not yet wired in the current codebase it is marked **PLANNED** with the defect id (D-1..D-7) and the plan task that delivers it. Do NOT read a **PLANNED** clause as a description of working behaviour. The **IMPLEMENTED TODAY** subsections describe behaviour confirmed against the live tree on 2026-06-10 (file:line cited). When in doubt, the cited code is the source of truth, not this prose.

Table of contents

1. [What “model access” means](#)
2. [Status legend & defect cross-reference](#)
3. [Step 1 — Supply or recognize an API key](#)
4. [Step 2 — Key presence makes a provider visible](#)
5. [Step 3 — Working models are dynamically discovered + verified \(LLMsVerifier\)](#)
6. [Step 4 — Per-provider exposure](#)
7. [Step 5 — The unified catalog \(GET /v1/catalog\)](#)
8. [Worked example: env var → provider visible → model usable](#)
9. [Behaviour when the verifier is disabled](#)
10. [Reference: per-provider env-var aliases](#)

1. What “model access” means

A model becomes usable to you through a **funnel** of conditions. The intended (target) rule is:

A provider’s **WORKING models** become available **iff** an API key for that provider is recognized (supplied directly, or read from `.env / api_keys.sh` / a shell export) **AND** the model is dynamically discovered and verified by **LLMsVerifier** (Verified == true, status verified, score $\geq$ the configured minimum).

This single rule is the goal of sub-program **SP1** (model-access refinement). The exposure of those working models as uniformly-named, individually-selectable targets is the goal of sub-program **SP2** (HelixAgent exposure). Both are planning-approved designs; their implementation status is tracked clause-by-clause below.

The funnel (target design, SP1 plan §3):

```
startup
├ secrets.LoadAPIKeys()           reads $HOME/api_keys.sh → else walks
├ config.Load()                  viper AutomaticEnv + per-provider Bi
├ keyrec.PresentProviders()       provider visible iff any of its env-
├ verifier.GetWorkingModels()    keep model iff present[provider] ^ V
└ expose                         CLI --list-models, server handlers,
```

2. Status legend & defect cross-reference

Label	Meaning
IMPLEMENTED TODAY	Confirmed present in the live tree on 2026-06-10 (file:line cited).
PARTIAL	The building block exists but is not wired into the user-facing path.
PLANNED	Designed in an approved plan but ABSENT from the current tree. Carries the defect id + delivering task.

Defects this guide depends on (roadmap ledger; see SP1/SP2/SP7 plans):

Defect	Summary	Current state (verified 2026-06-10)	Delivered by
D-1	<code>/v1/completion/models</code> returns a hardcoded 3-model list	PRESENT — <code>submodules/helix_agent/internal/handlers/completion.go:406</code> hardcodes <code>deepseek-coder/claude-3-sonnet-20240229/gemini-pro</code>	SP2 P2.0
D-2		PRESENT — <code>helix_code/cmd/cli/main.go:1361/1392</code>	SP1 P1.3.2

Defect	Summary	Current state (verified 2026-06-10)	Delivered by
	CLI <code>--list-models</code> shows failed/pending models as available		
D-3	<code>secrets.LoadAPIKeys</code> is dead — never called in a prod startup path	PRESENT — defined at <code>helix_code/internal/secrets/loader.go:30</code> , no prod call-site	SP1 P1.1.1
D-4	Working-model filter (Verified $\wedge$ <code>score ≥ min</code>) loaded but never applied	PRESENT — <code>helix_code/internal/verifier/adapter.go:175</code> <code>GetMinAcceptableScore</code> exists, never invoked in a listing path	SP1 P1.3.1
D-5	Hardcoded provider model lists (CONST-036)	PRESENT — <code>openai_provider.go/anthropic_provider.go</code> literal slices	SP1 P1.3.4

3. Step 1 — Supply or recognize an API key

You can make a key available to HelixCode in any of three ways. The intended precedence (SP1 DECISION-1, recommended **gap-fill**: an already-exported shell var is NOT overwritten by a file) is **PLANNED**; the loader mechanism itself is **IMPLEMENTED TODAY** but not yet invoked at startup.

3a. Shell export (recommended for ad-hoc / CI)

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

- **IMPLEMENTED TODAY**: config binds per-provider env vars via Viper `AutomaticEnv + SetEnvPrefix("HELIX")` and explicit `BindEnv` for the 8 cloud providers — `helix_code/internal/config/config.go:307-345`. A shell export of a bound variable is read at config load.

3b. `api_keys.sh` in your home directory

```
# $HOME/api_keys.sh
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
```

- **PARTIAL**: the loader that reads `$HOME/api_keys.sh` exists — `helix_code/internal/secrets/loader.go:30` (`LoadAPIKeys()` → reads `api_keys.sh` first). It does **not** log values (CONST-042-clean).
- **PLANNED (D-3)**: `LoadAPIKeys()` is **not yet called** from any binary's `main()` (verified: only its own definition + doc-comments reference it in the non-test tree). Until SP1 task **T1.1.1** wires it as the first statement of `cmd/server/main.go` and `cmd/cli/main.go`, an `api_keys.sh`-only key is **not** recognized — you must source `~/api_keys.sh` yourself so it reaches the shell environment.

3c. .env file (walked up from the working directory)

```
# .env
ANTHROPIC_API_KEY=sk-ant-...
```

- **PARTIAL:** `LoadAPIKeys()` falls back to a walked-up `.env` (`loader.go` `findEnvFile/loadFromEnv`).
- **PLANNED (D-3):** same wiring gap as 3b — auto-load at startup arrives with **T1.1.1**.

CONST-042 (no-secret-leak): never commit a real key. `.env` is gitignored mode 0600; `.env.example` carries placeholders only. The loader never prints key values at any log level.

4. Step 2 — Key presence makes a provider visible

The intended rule: a provider appears **only if** at least one of its env-var aliases is set to a non-placeholder value. A user with only an Anthropic key must NOT see OpenAI, Gemini, etc.

- **PLANNED (no-key-presence-gate gap, SP1 §1):** today there is **no** key-presence gate in front of the listing path, and the provider factory (`helix_code/internal/llm/factory.go`) builds **one** provider by config. Type rather than enumerating every key-present provider.
- **PLANNED:** SP1 adds:
 - `helix_code/internal/llm/keyrecognition.go` — `PresentProviders()` `map[string]bool` + the multi-alias `{provider → []envVarAlias}` table + `isPlaceholder()` (task **T1.1.2**). **ABSENT today** (file does not exist). The alias table is lifted, decoupled, from `helix_agent/SupportedProviders.EnvVars(submodules/helix_agent/internal/verifier/provider_types.go)` per §11.4.74.
 - `BuildPresentProviders(configs, present)` in `factory.go` — builds **every** key-present provider, not one (task **T1.2.2**).

See [§10](#) for the alias table.

5. Step 3 — Working models are dynamically discovered + verified (LLMsVerifier)

LLMsVerifier is the single source of truth for model metadata, verification status, and scoring (CONST-036). No hardcoded model lists are permitted as the user-facing source.

What exists today

- **IMPLEMENTED TODAY:** `helix_code/internal/verifier/adapters.go`
 - `GetVerifiedModels(ctx)` (`adapters.go:183`) — gated on `IsEnabled()`, served from cache → live → stale → fallback, then `filterByProviderConfig`.
 - `GetMinAcceptableScore()` (`adapters.go:175`) — returns the configured minimum (default 6.0).

- `VerifiedModel` carries the “working” fields: `Verified` bool, `VerificationStatus` string, `OverallScore` float64, `ContextSize`, `LastVerified` (`verifier/types.go`).
- The verifier client makes **real HTTP** calls (`verifier/client.go`) — this is real discovery, not a simulation.

The gap (D-2, D-4)

- **PARTIAL / PLANNED:** the working-model predicate exists only in *pieces*. `GetMinAcceptableScore()` is **never invoked** in any listing path, and `filterByProviderConfig` drops only providers with `Enabled == false` — it applies **no** `Verified/status/score` filter and **no** key-presence gate.

- **PLANNED (D-4):** SP1 task **T1.3.1** adds `GetWorkingModels(ctx, present)` to `adapter.go`, keeping a model iff:

```
present[m.Provider]                                (key-presence gate – NEW)
^ m.Verified == true
^ m.VerificationStatus == "verified"
^ m.OverallScore >= GetMinAcceptableScore()
^ config.Providers[m.Provider].Enabled != false
```

`GetWorkingModels` is **ABSENT today** (verified: `adapter.go` has `GetVerifiedModels + GetMinAcceptableScore` but no `GetWorkingModels`).

- **PLANNED (D-2):** until then, the CLI lists models straight from `GetVerifiedModels` and the printer (`cmd/cli/main.go:1392 printVerifiedModels`) renders failed/pending/rate-limited rows as if available. Treat the current `--list-models` output as a raw catalog, NOT a working-model list.

Freshness guarantees (target): verified within 24h (CONST-037), status reflected within 60s / poll ≤ 60 s (CONST-038), honoured via the adapter cache TTL.

6. Step 4 — Per-provider exposure

`HelixAgent` (`submodules/helix_agent`) exposes the runtime providers over Gin `/v1` routes.

- **IMPLEMENTED TODAY:** `GET /v1/providers` (`submodules/helix_agent/internal/router/router.go ~:773`) lists providers, reading live `GetCapabilities().SupportedModels` from the runtime `services.ProviderRegistry` (`internal/services/provider_registry.go`).
- **IMPLEMENTED TODAY (but a bluff):** `GET /v1/completion/models` (`internal/handlers/completion.go:406`) returns a **hardcoded** 3-model list (**D-1**, CONST-036 / BLUFF-002 violation). Do not rely on it as an authoritative model list.
- **PLANNED (D-1):** SP2 task **T2.0.2** rewrites `CompletionHandler.Models` to return the live verifier-sourced (`Verified==true`) list joined with the registry — never a literal map.

HelixLLM is currently a gated provider named "helixllm" (enabled by USE_HELIX_LLM=true, provider_registry.go:750-764); **PLANNED (SP2 T2.2.4):** promote it to a first-class root entry (helixllm + helixllm/<model>).

7. Step 5 — The unified catalog (GET /v1/catalog)

The intended single entry point that joins **ensemble + HelixLLM + every provider + every verified model** into one list of uniformly-named, individually-selectable targets.

- **PLANNED (SP2 P2.1):** GET /v1/catalog is **ABSENT today** (verified: no /v1/catalog route in router.go, and submodules/helix_agent/internal/catalog/ does not exist). The root path itself is an operator decision (SP2 OP-1; default GET /v1/catalog).
- **PLANNED — naming scheme** (SP2 §0, confirmed against existing conventions). The request asks for the form ensemble//helixllm//<provider>/<model>; the catalog grammar that realises it is:

Target class	Name form	Example
AI-debate ensemble (aggregate)	ensemble	ensemble
Ensemble preset	ensemble/<preset>	ensemble/ confidence_weighted
HelixLLM root	helixllm	helixllm
HelixLLM model	helixllm/<model>	helixllm/helixllm- default
Provider (individual)	<provider>	anthropic, groq
Provider working model	<provider>/ <model_id>	anthropic/claude-3- sonnet-20240229
Already-namespaced model id	<provider>/ <vendor>/<model>	openrouter/x-ai/ grok-4

A model entry is emitted **only** for DiscoveredModel.Verified == true — never from a static SupportedModels slice, never hardcoded.

- **PLANNED — selector resolution** (SP2 T2.1.4): the existing free-text model field on a completion request resolves any catalog Name to the right target. An unknown selector returns 400 with the candidate list (no silent guess; §11.4.6 / §11.4.105).
-

8. Worked example: env var → provider visible → model usable

This is the **target** end-to-end flow (the SP1 E2E case, tests/e2e/challenges/sp1_working_models). Steps marked **(today)** already behave as shown; steps marked **(after SP1/SP2)** require the planned wiring.

```
# 1. Supply exactly ONE provider key.
export ANTHROPIC_API_KEY="sk-ant-..."           # (today) bound via
config.go:307-345

# 2. (after SP1 T1.1.1) startup auto-loads api_keys.sh / .env too
      - gap-fill precedence,
#     so this shell export is NOT overwritten by a file value.

# 3. List models.
./bin/cli --list-models
```

Intended output **after SP1**:

```
# Only Anthropic appears (key-presence gate, Step 2), and only its VERIFIED
# NO openai/gemini/etc rows, NO failed/pending rows.
anthropic/claude-3-5-sonnet-20241022    ✓ verified    (context: 200000, score: 1.0)
anthropic/claude-3-sonnet-20240229     ✓ verified    (context: 200000, score: 1.0)
```

Intended output **today** (current tree, before SP1): the CLI prints the whole verifier catalog via `GetVerifiedModels`, including failed/pending rows and providers you have no key for (D-2, D-4). So treat today's list as unfiltered.

```
# 4. Use a model — make a REAL generation call.
./bin/cli generate --model anthropic/claude-3-5-sonnet-20241022 \
  --prompt "What is 2+2?"
# (today) handleGenerate calls provider.Generate / GenerateStream
      directly — real HTTP, real output
# (BLUFF-001 is resolved). The answer is real model output, never
      a simulated string.
```

The anti-bluff acceptance bar (§11.4.107): a model is only “working” once a **real generate returns real output** — listing it is necessary but not sufficient.

9. Behaviour when the verifier is disabled

`HELIX_VERIFIER_ENABLED=false` (default per `.env.example`) removes the only authoritative “working” signal.

- **PLANNED (SP1 DECISION-2)**: the `working-models` command **MUST NOT** present the verifier's `FallbackModels` as “working” — doing so is a CONST-035 / §11.4 PASS-bluff and is explicitly rejected. The recommended behaviour: return an **empty** list plus an honest notice (e.g. “enable `HELIX_VERIFIER_ENABLED` to see working models”), with a per-provider live `GetModels()` path available only behind an explicit `--include-unverified` flag (labelled `unverified`).
 - **Current tree (D-2)**: `cmd/cli/main.go:1387` falls back to `verifier.FallbackModels` and prints it — the exact bluff SP1 T1.3.3 removes. Until then, do not interpret the disabled-verifier list as verified.
-

10. Reference: per-provider env-var aliases

The intended multi-alias recognition table (SP1 task T1.1.2, lifted decoupled from `helix_agent_provider_types.go SupportedProviders.EnvVars`). **PLANNED** — `keyrecognition.go` is ABSENT today; the table below documents the target. Setting **any** alias for a provider (to a non-placeholder value) makes it present.

Provider	Recognized env-var aliases (target)
openai	OPENAI_API_KEY
anthropic	ANTHROPIC_API_KEY, CLAUDE_API_KEY
gemini	GEMINI_API_KEY, GOOGLE_API_KEY
deepseek	DEEPSEEK_API_KEY
groq	GROQ_API_KEY
mistral	MISTRAL_API_KEY
xai	XAI_API_KEY
openrouter	OPENROUTER_API_KEY

The authoritative alias set is the `helix_agent SupportedProviders.EnvVars` map; the table above mirrors the 8 providers explicitly bound in `helix_code/internal/config/config.go:338-345`. Local providers (Ollama, Llama.cpp) need no key and are handled by their own discovery paths. When `keyrecognition.go` lands, that file — not this table — is the source of truth.

11. What is working today vs planned (summary)

Working today (verified 2026-06-10): - Real LLM generation via the CLI (BLUFF-001 resolved — `handleGenerate` calls `provider.Generate`). - Config binds the 8 cloud-provider env vars (`config.go:307-345`); a shell export is read at config load. - The `secrets.LoadAPIKeys` loader (reads `api_keys.sh` then `.env`, CONST-042-clean) — exists but is **not invoked at startup**. - The verifier adapter makes real HTTP discovery (`GetVerifiedModels`, `GetMinAcceptableScore`) and carries the `Verified/status/score` fields. - GET `/v1/providers` lists runtime providers from live capabilities.

Planned (ABSENT or unwired today): - D-3: auto-loading `api_keys.sh/.env` at startup (T1.1.1). - Key-presence gate + multi-alias `keyrecognition.go` + key-present provider enumerator (T1.1.2 / T1.2.x). - D-4: `GetWorkingModels` applying `Verified ^ status ^ score ≥ min` (T1.3.1). - D-2: CLI/server listing only working models, dropping failed/pending/no-key providers (T1.3.2). - D-5: de-hardcoding provider model lists (T1.3.4). - D-1: verifier-sourced `/v1/completion/models` (SP2 T2.0.2). - The unified GET `/v1/catalog+ensemble/helixllm/<provider>/<model>` naming + selector resolution (SP2 P2.1). - HelixLLM promotion to a first-class root (SP2 T2.2.4).

Sources verified 2026-06-10

- Plans: `docs/superpowers/specs/plans/2026-06-10-SP1-model-access-plan.md`, ... - SP2-helixagent-exposure-plan.md.

- Live tree: helix_code/internal/secrets/loader.go:30, helix_code/internal/verifier/adapter.go:175,183, helix_code/internal/config/config.go:307-345, helix_code/cmd/cli/main.go:1361,1387,1392, submodules/helix_agent/internal/handlers/completion.go:406, submodules/helix_agent/internal/router/router.go (no /v1/catalog), keyrecognition.go / internal/catalog/ confirmed ABSENT.